

I replaced Mailchimp with a Rust Worker and a self-hosted mail server

Emil Lindfors | February 18, 2026

Update, August 2026. Most of this still stands, but three things in it are now wrong: Stalwart 0.16 deleted the management API this post uses, subscribing is double opt-in, and the newsletter is no longer fanned out by the mail server. I have corrected the descriptions below. The story of *why* each of those changed – including the fact that the `List-Unsubscribe` headers praised here were unsigned and therefore useless for the entire life of this post – is in the follow-up: [My newsletter promised one-click unsubscribe and answered every one with a 400](#).

I wanted a newsletter for my blog. The requirements were simple: let people subscribe, send them posts when I publish, let them unsubscribe. That's it.

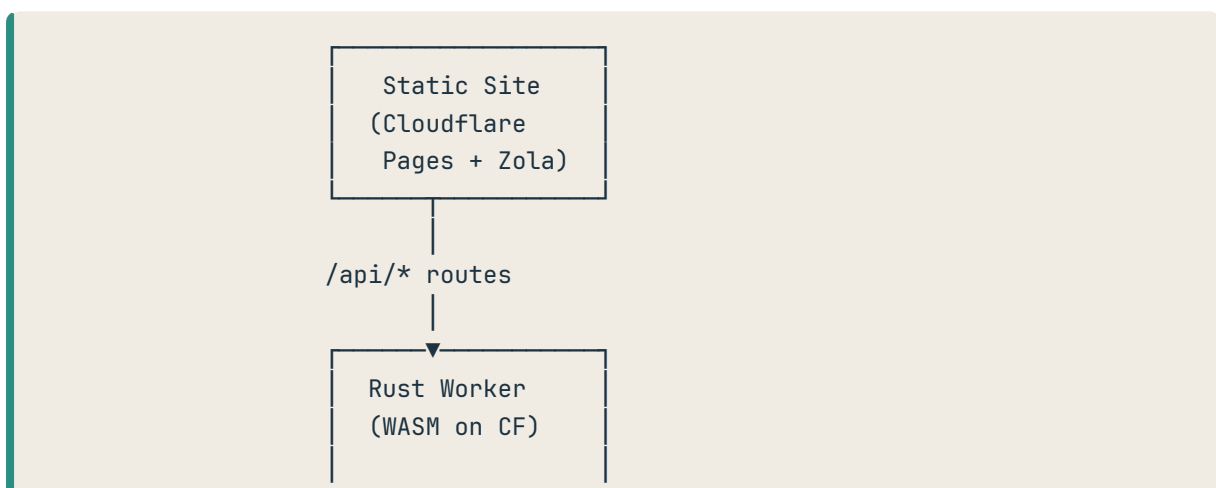
Every newsletter platform I looked at wanted \$10-30/month, required me to hand over my subscriber list, injected their branding, and came with dashboards I'd never use. For a personal blog that publishes once or twice a month, this felt absurd.

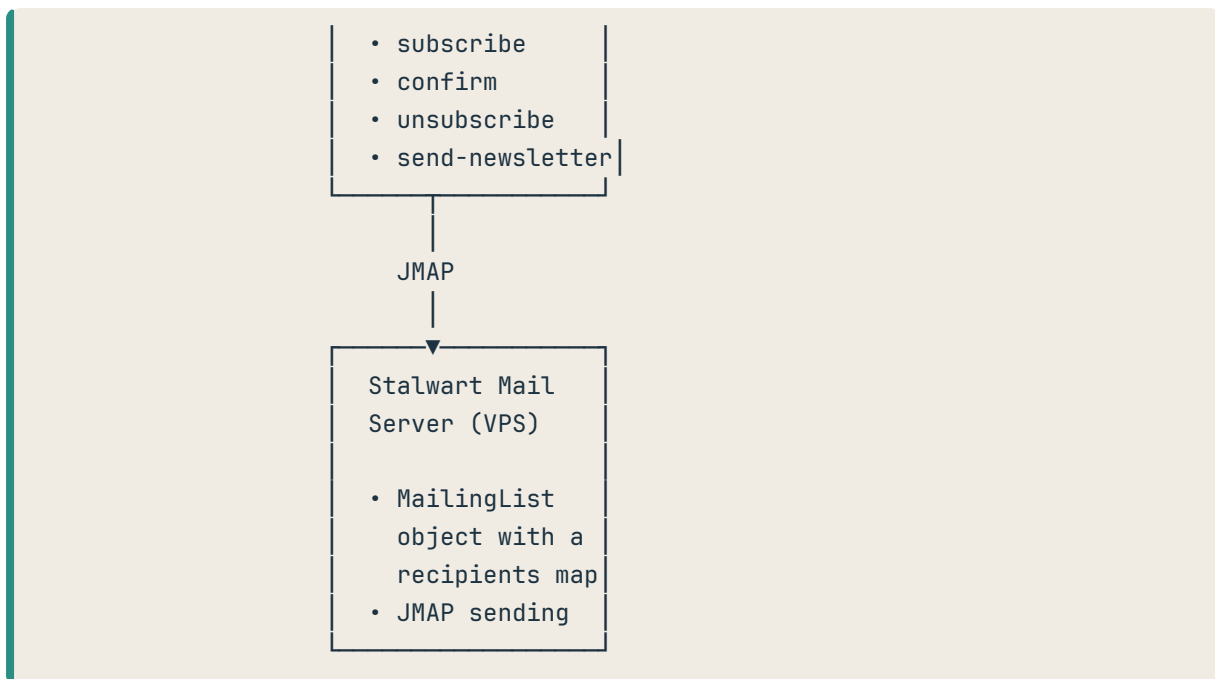
So I built my own. The entire system is ~700 lines of Rust, runs on Cloudflare's free tier, and uses my existing self-hosted mail server for delivery. Monthly cost: \$0 on top of infrastructure I already had.

Here's how it works.

The architecture

The setup has three components:





1. **Zola static site** on Cloudflare Pages – generates the blog, serves HTML
2. **Rust Cloudflare Worker** – handles `/api/*` routes for subscribe, unsubscribe, and sending
3. **Stalwart mail server** – self-hosted on a VPS, acts as both the subscriber store and the delivery engine

The key insight is that **Stalwart's mailing list eliminates the need for a database entirely**. A Stalwart MailingList object has a `recipients` map, `{"someone@example.com": true}`. That map is my subscriber list. No database. No subscriber table. No sync jobs.

It held up better than I expected. When I added double opt-in six months later, the obvious cost was a pending-subscribers table – and it turned out not to need one. That is the [follow-up post](#).

The Cloudflare Worker

The worker is built with **workers-rs**, the Rust SDK for Cloudflare Workers. It compiles to WASM and runs on Cloudflare's edge network.

The `Cargo.toml` is minimal:

```
[dependencies]
worker = "0.7"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
pulldown-cmark = { version = "0.12", default-features = false, features = ["html"] }
```

Four dependencies. `pull-down-cmark` is there because the worker renders newsletter mark-down to HTML at send time.

Subscribe

When someone enters their email on my site, the form posts to `/api/subscribe`. The worker validates the email, rate-limits the caller, and mails a signed confirmation link. Pressing the button in that mail posts to `/api/confirm`, and only then does the address reach the list:

```
/// Add or remove one address in the mailing list's `recipients` map.
async fn jmap_set_recipient(
    cfg: &ListConfig,
    email: &str,
    subscribe: bool,
) → std::result::Result<(), String> {
    let value = if subscribe {
        serde_json::Value::Bool(true)
    } else {
        serde_json::Value::Null
    };

    let mut patch = serde_json::Map::new();
    patch.insert(format!("recipients/{}", json_pointer_escape(email)), value);
    // ... wrap in an `update` for x:MailingList/set and send it over JMAP
}
```

That's it. One `x:MailingList/set` call setting `recipients/<addr>` to `true`, or to `null` to remove it. Unsubscribe is the same call with the other value.

The original version of this post did a single `addItem` PATCH against Stalwart's REST management API at `/api/principal/{list_id}`, and said "no confirmation email, no double opt-in dance (I should probably add that eventually)". Both of those are gone now: 0.16 deleted the REST API, and the double opt-in dance turned out to be worth doing.

Sending newsletters via JMAP

This is where it gets interesting. Instead of SMTP, I use **JMAP** (JSON Meta Application Protocol) to send emails. JMAP is a modern, stateful, JSON-based protocol designed to replace the IMAP/SMTP combo. Stalwart has full JMAP support.

Sending a newsletter is a single JMAP request with two method calls batched together:

```
let body = serde_json::json!({
    "using": [
        "urn:ietf:params:jmap:core",
        "urn:ietf:params:jmap:mail",
        "urn:ietf:params:jmap:submission"
    ]
})
```

```

],
"methodCalls": [
  ["Email/set", {
    "accountId": account_id,
    "create": {
      "draft": {
        "from": [{ "name": "Emil Lindfors", "email": from }],
        "to": [{ "email": "newsletter@lindfors.no" }],
        "subject": subject,
        "header:List-Unsubscribe:asRaw":
          " <https://lindfors.no/api/unsubscribe>",
        "header:List-Unsubscribe-Post:asRaw":
          " List-Unsubscribe=One-Click",
        "htmlBody": [{ "partId": "html", "type": "text/html" }],
        "bodyValues": {
          "html": { "value": html_body }
        }
      }
    }
  }, "0"],
  ["EmailSubmission/set", {
    "accountId": account_id,
    "create": {
      "send": {
        "identityId": identity_id,
        "emailId": "#draft",
        "envelope": {
          "mailFrom": { "email": from },
          "rcptTo": [{ "email": "newsletter@lindfors.no" }]
        }
      }
    },
    "onSuccessDestroyEmail": ["#send"]
  }, "1"]
]
});

```

The first call (`Email/set`) creates the email as a draft. The second (`EmailSubmission/set`) submits it for delivery, referencing the draft with `#draft`. The `onSuccessDestroyEmail` cleans up the draft after sending. All in one HTTP request.

That request went to `newsletter@lindfors.no`, the mailing list address, and Stalwart fanned it out to every subscriber. It does not any more. The worker reads the `recipients` map and runs the request above once per address, because the `List-Unsubscribe` header has to name a different URL for each recipient and a fanned-out message can only carry one. Why that turned out to be forced rather than a preference is the [follow-up](#).

Note the `List-Unsubscribe` and `List-Unsubscribe-Post` headers. They are supposed to make email clients show a native “Unsubscribe” button instead of leaning towards the spam button. Mine did nothing at all for six months, because RFC 8058 requires both headers to be named in the DKIM signature’s `h=` tag and Stalwart’s default signs five headers, none of which are these.

The Stalwart side

Stalwart is a mail server written in Rust. I run it on a small VPS. It handles SMTP, IMAP and JMAP. Up to 0.16 it also had a REST management API, which is where the original version of this post did its subscriber management; 0.16 deleted it, and everything now goes over JMAP.

The only Stalwart configuration specific to newsletters is a mailing list. As of 0.16 that is a `MailingList` object, id `e`, with:

- An email address (`newsletter@lindfors.no`)
- A `recipients` map, `{"someone@example.com": true}`, holding the subscribers

Stalwart will still deliver a copy to every recipient if you mail the list address, and that is a perfectly good delivery mechanism if you do not need per-recipient unsubscribe links. I do, so the worker sends individually instead.

I created the list through Stalwart’s admin interface. No special configuration files. The worker manages members with `x:MailingList/set`, which needs the `sysMailingListUpdate` permission on the account it authenticates as.

The send workflow

I write blog posts in markdown with Zola. When I want to send a post as a newsletter:

1. Generate the newsletter markdown:

```
site-tools newsletter gen content/blog/my-post/index.md
```

This extracts the post body, strips markup that only makes sense on the web (math blocks, figures), and writes a clean markdown file to `static/newsletter/my-post.md` with YAML frontmatter:

```
---
title: "My Post Title"
date: "2026-02-10"
description: "Post description"
url: "https://lindfors.no/blog/my-post/"
---
```

2. Deploy the site so the `.md` file is accessible at `https://lindfors.no/newsletter/my-post.md`.

3. Send:

```
site-tools newsletter send my-post
```

This reads the admin key from `.env` and calls the worker:

```
curl -s --fail-with-body -X POST "https://lindfors.no/api/send-newsletter" \
  -H "Authorization: Bearer $ADMIN_KEY" \
  -H 'Content-Type: application/json' \
  -d '{"slug":"my-post"}'
```

The admin key started out in the query string, where it ended up in logs. It is a bearer header now. `--fail-with-body` is there because `curl -s` exits 0 on an HTTP 500, so for a while the CLI reported failed sends as successes.

The worker fetches the markdown from my site, parses the frontmatter, renders the body to HTML with `pulldown-cmark`, wraps it in an email template, and sends one copy per recipient via JMAP.

The email template is inline in the worker – hardcoded HTML with inline styles (because email clients). No template engine, no CSS framework. It looks clean and renders consistently across Gmail, Apple Mail, and Outlook.

What this costs

Component	Cost
Cloudflare Pages	Free
Cloudflare Worker	Free (100k requests/day)
Stalwart on VPS	~\$5/month (shared with other services)
Domain	~\$10/year

Compare this to Mailchimp (13/month for 500 subscribers), ConvertKit (29/month), or Substack (10% of paid subscriptions). For a personal blog newsletter, the economics aren't even close.

What's missing

This is not a replacement for Mailchimp if you need marketing features. Things I don't have:

- **Analytics** – No open tracking, no click tracking. I don't care about this, but you might.
- **Bounce handling** – Stalwart handles bounces at the SMTP level, but I don't automatically remove bouncing addresses from the list.

- **Pretty email editor** – I write markdown. The template is hardcoded Rust. This is a feature, not a bug.
- **Scheduling** – I run a shell script when I want to send. No scheduled sends.
- **Any list longer than 45 addresses.** Since the switch to per-recipient sending, each message is a subrequest and Workers caps those per invocation. Past 45 the send is refused rather than truncated, and getting further needs batching or a queue.

For a personal blog with a handful of subscribers who actually want to read what I write, none of these are real problems.

Double opt-in used to be on this list. It isn't any more, and adding it did not cost me a database.

Workers-rs tips

A few things I learned building this:

Route order matters. In workers-rs, register GET routes before POST routes for the same path. I had the unsubscribe GET page and POST handler on the same `/api/unsubscribe` path and the order of registration determined which matched.

Headers aren't mutable the way you'd expect. `Headers::new()` methods take `&self`, not `&mut self`. You don't need `let mut headers`.

No `.url()` on `RouteContext`. If you need the request URL (for query parameters), use `req.url()?` rather than trying to get it from the route context.

Release profile matters for WASM size. I use aggressive optimization:

```
[profile.release]
lto = true
strip = true
codegen-units = 1
```

Why not just use Substack?

Substack is free for free newsletters. Here's why I didn't:

1. **I already have a blog.** My posts are markdown in a git repo, rendered by Zola, deployed to Cloudflare. I don't want a second copy of my content on someone else's platform.
2. **I already run a mail server.** Stalwart was set up for personal email. The newsletter is just one mailing list on it.
3. **Ownership.** My subscriber list is a JSON map in my mail server. I can export it with one API call. No platform lock-in, no "download your data" request, no worrying about a service shutting down or changing terms.

4. **It's a fun project.** The whole thing took an afternoon. It's 700 lines of Rust that I fully understand and can modify. That has value.

On that last point: it is about 2,200 lines now, after double opt-in, rate limiting, signed tokens and 30 tests. Most of that growth was the difference between a thing that works for me and a thing that can be pointed at strangers.

The newsletter stack

- **API:** [workers-rs](#) (Rust, compiled to WASM)
- **Mail server:** [Stalwart](#) (Rust) on a VPS
- **Protocol:** JMAP, for both sending and subscriber management
- **Markdown rendering:** [pulldown-cmark](#) (in the worker, at send time)
- **DNS:** Cloudflare (SPF, DKIM, DMARC records)

The code for the worker is [on GitHub](#). If you're running Stalwart and want to try this, the setup is straightforward – a worker, a mailing list, and a few environment variables. Read [the follow-up](#) before you copy the unsubscribe handling out of this one.